TATA TECHNOLOGIES | INNOVENT-27 | AI AT THE EDGE | AEROSPACE TRACK

AeroTwin — System Architecture Document

Technical Design Specification | Team FELONS | InnoVent-27

ARCHITECTURE OVERVIEW

AeroTwin follows a clean three-tier client-server architecture augmented with a dedicated real-time communication layer. The system is designed around five core engineering principles: low-latency streaming, modularity, edge-readiness, offline operability, and clear separation of concerns between simulation, processing, and presentation layers.

DESIGN PHILOSOPHY

Every architectural decision in AeroTwin is made with a future hardware migration in mind. The simulation engine at the bottom of the stack can be replaced by a real IoT sensor feed without modifying a single line of code in the fatigue engine, ML pipeline, WebSocket server, or React dashboard above it. This clean boundary is the system's most important architectural property.

FIVE-LAYER SYSTEM ARCHITECTURE

AeroTwin's full pipeline is organised across five distinct layers, each with a single well-defined responsibility:

Layer 1 — Sensor Data Layer

Responsibility: Generate or ingest time-series sensor telemetry for each monitored engine component.

- Current implementation: A Python-based physics-informed simulation engine running as a background thread inside the Flask server. Generates temperature (°C), vibration (mm/s), and RPM readings per component per simulation tick, with calibrated noise, flight-phase variation, and non-linear degradation curves based on NASA C-MAPSS model parameters.
- Future implementation (Stage 2): Replace simulation loop with serial/USB read from Arduino-attached MEMS accelerometer (MPU-6050) and thermocouple (K-type + MAX6675 ADC). Zero changes required to any layer above.
- Future implementation (Stage 3): NVIDIA Jetson Nano with industrial MEMS sensors communicating via MQTT broker. Model inference runs on-device (true Edge AI).
- Key design decision: Sensor readings are emitted as structured JSON objects per tick: { component_id, flight_hour, temperature, vibration, rpm, timestamp }. This schema is fixed and shared by all layers.

Layer 2 — Processing & Intelligence Layer

Responsibility: Transform raw sensor readings into health scores, fatigue curves, and RUL predictions.

- Fatigue Accumulation Engine: Consumes each sensor reading and applies the multi-factor fatigue formula: fatigue_delta = (T_factor^1.4 × V_factor^1.8 × RPM_factor^1.2) × component_sensitivity. Updates a running cumulative fatigue score per component. Health score = max(0, 100 − cumulative_fatigue). Runs synchronously within each simulation tick.
- ML Inference Pipeline (scikit-learn): A pre-trained Random Forest Regressor takes engineered features (rolling mean of vibration over last 10 ticks, rate of temperature rise, vibration slope gradient, inter-sensor correlation coefficient) and outputs: predicted_rul (flight hours remaining) and prediction_confidence (based on inter-tree variance in the forest). Inference runs in < 5ms per tick on standard laptop hardware.
- Rule-Based Alert Engine: Evaluates predicted_rul against tiered thresholds. RUL > 500hrs → GREEN / Healthy. RUL 100–500hrs → AMBER / Schedule inspection. RUL < 100hrs → RED / Priority inspection. RUL < 50hrs → CRITICAL / Ground aircraft. Alert payloads include component_id, severity, recommended_action, and estimated_time_to_service.
- Anomaly Detector: A secondary statistical layer computes rolling z-scores on the vibration and temperature streams. z-score > 3.5 triggers an immediate anomaly flag independent of the RUL model — catching sudden step-change faults that the gradual-degradation model may not flag fast enough.

▸ Layer 3 — Communication Layer

Responsibility: Deliver processed data from backend to frontend with minimum latency and maximum reliability.

- Protocol: Socket.IO over WebSocket (with automatic HTTP long-polling fallback for resilience). Persistent bidirectional connection maintained between Flask-SocketIO server and all connected React clients.
- Event schema: Three event types emitted per tick — sensor_update (raw readings for historical log), health_update (fatigue scores + health percentages per component), alert_update (any new alerts or anomaly flags). Each event carries a server_timestamp for client-side latency measurement.
- Measured latency: < 100ms end-to-end from sensor reading generation to React state update on localhost. In a LAN deployment, < 150ms. In a real aircraft deployment (satellite link), the architecture supports batching multiple ticks into a single transmission to manage bandwidth.
- Bidirectional capability: Client can send control events upstream — inject_anomaly (triggers fault in simulator), advance_time (jumps simulation forward), reset_simulation (restores all components to baseline). This enables the live demonstration's interactive anomaly injection feature.

▸ Layer 4 — Application & API Layer

Responsibility: Expose structured REST endpoints for historical data and manage application state.

- Framework: Python Flask with Flask-SocketIO extension. Single-process application with SocketIO managing the background simulation thread via eventlet/gevent async worker.
- REST API Endpoints: GET /api/health — returns current health snapshot for all components. GET /api/history?component=<id>&hours=<n> — returns last n hours of sensor readings for a component from SQLite. GET /api/rul — returns current ML model RUL prediction and confidence per component. POST /api/anomaly — injects a specified fault type into the simulation. GET /api/reset — resets simulation to baseline state.

- Database interaction: SQLAlchemy ORM (or raw sqlite3 for simplicity) handles async-safe reads/writes to SQLite. Every sensor reading is persisted before being processed, ensuring historical completeness even if the client disconnects mid-session.
- CORS configuration: Flask-CORS enabled for localhost:3000 (React dev server) during development; configurable for production deployment.

▸ Layer 5 — Presentation Layer

Responsibility: Render all data as a professional, intuitive, real-time interactive dashboard.

- Framework: React.js (functional components, hooks). State management via useState + useEffect with Socket.IO event listeners updating component state on every incoming event — no full page refresh required.
- 3D Visualization: Three.js renders a schematic engine cross-section with three labeled component meshes (turbine blade, bearing, compressor). Each mesh's material color interpolates smoothly between GREEN (0x00C853) → AMBER (0xFF6D00) → RED (0xD50000) as the component's health score decreases. Hovering a component surfaces a tooltip showing current sensor readings and RUL.
- Live Charts: Recharts LineChart components render three overlapping series per component: temperature trend, vibration trend, and fatigue score — all updating on every sensor_update event. X-axis shows simulated flight hours; Y-axis auto-scales. Chart history is capped at 200 data points (rolling window) to prevent memory growth during long sessions.
- Alert Panel: A fixed sidebar lists all active and historical alerts in reverse chronological order. Critical alerts pulse with a CSS animation; all alerts show component ID, severity badge, recommended action, and timestamp. The panel persists across the session without requiring database reads.
- Historical Log Modal: A scrollable table pulls data from GET /api/history and renders the full sensor reading history for any selected component — allowing judges to review the complete degradation trajectory that led to the current state.

DATA FLOW DIAGRAM — STEP BY STEP

The following table traces the complete journey of a single data point from generation to dashboard render:

Step	Action	Input	Output
1	Simulation tick fires (every 2s)	Current flight_hour, previous sensor state	Raw JSON reading: {temp, vibration, rpm}
2	SQLite write	Raw JSON reading	Persisted sensor_readings table row
3	Fatigue engine processes reading	Raw reading + previous fatigue score	Updated health_score per component (0–100)
4	Feature engineering	Last 10 readings per component	12 engineered ML features per component
5	Random Forest inference	12 engineered features	predicted_rul +

			confidence_interval
6	Alert engine evaluates RUL	predicted_rul per component	Alert payload (severity, action) if threshold crossed
7	Anomaly z-score check	Rolling vibration + temp z-scores	Anomaly flag if z > 3.5
8	SocketIO emit — 3 events	health_update, sensor_update, alert_update	JSON payloads dispatched to all connected clients
9	React state update	Incoming SocketIO events	Component re-renders with new data (no full reload)
10	Three.js re-render	Updated health_score per component	Mesh color interpolation, smooth visual transition
11	Recharts chart append	New sensor data point	Chart series extended, oldest point dropped (rolling 200)
12	Alert panel update	New alert payload (if any)	Alert card prepended to sidebar with severity animation

DATABASE SCHEMA — SQLITE

AeroTwin uses three tables in a single SQLite file (aerotwin.db):

▸ Table: sensor_readings

Column	Type / Constraint	Description
id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique row identifier
component_id	TEXT NOT NULL	turbine_blade / bearing / compressor
flight_hour	INTEGER NOT NULL	Simulated flight hour at time of reading
temperature	REAL NOT NULL	Temperature in °C
vibration	REAL NOT NULL	Vibration in mm/s
rpm	REAL NOT NULL	Engine RPM
timestamp	DATETIME DEFAULT CURRENT_TIMESTAMP	Server time of reading

▸ Table: health_snapshots

Column	Type / Constraint	Description

id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique row identifier
component_id	TEXT NOT NULL	Component identifier
flight_hour	INTEGER NOT NULL	Simulated flight hour
fatigue_score	REAL NOT NULL	Cumulative fatigue (0–100)
health_score	REAL NOT NULL	100 − fatigue_score
predicted_rul	REAL NOT NULL	ML model RUL estimate (flight hours)
confidence	REAL NOT NULL	Prediction confidence (0–1)
timestamp	DATETIME DEFAULT CURRENT_TIMESTAMP	Server time of snapshot

▸ Table: maintenance_log

Column	Type / Constraint	Description
id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique row identifier
component_id	TEXT NOT NULL	Component that triggered the alert
severity	TEXT NOT NULL	GREEN / AMBER / RED / CRITICAL
recommended_action	TEXT NOT NULL	Human-readable maintenance instruction
predicted_rul_at_alert	REAL NOT NULL	RUL value that triggered this alert
anomaly_flag	INTEGER DEFAULT 0	1 if triggered by z-score anomaly detector
timestamp	DATETIME DEFAULT CURRENT_TIMESTAMP	Time alert was generated

REST API REFERENCE

All REST endpoints are served by the Flask backend on port 5000:

Method	Endpoint	Parameters	Response
GET	**/api/health**	None	JSON: current health_score, fatigue_score, predicted_rul, confidence per component
GET	**/api/history**	component (str), hours (int)	JSON array of sensor_readings for specified component and

			time window
GET	**/api/rul**	None	JSON: predicted_rul and confidence interval per component
GET	**/api/alerts**	limit (int, default 50)	JSON array of maintenance_log entries, most recent first
POST	**/api/anomaly**	Body: { type: vibration|thermal|rpm }	Injects specified fault into simulator; returns { status: injected }
POST	**/api/reset**	None	Resets simulation to baseline; clears in-memory state (DB retained)
GET	**/api/components**	None	JSON: list of monitored component IDs and their display names

WEBSOCKET EVENT REFERENCE

Socket.IO events emitted by the Flask-SocketIO server on each simulation tick:

Event Name	Direction	Payload Schema
sensor_update	Server → Client	{ component_id, flight_hour, temperature, vibration, rpm, server_timestamp }
health_update	Server → Client	{ component_id, health_score, fatigue_score, predicted_rul, confidence, severity }
alert_update	Server → Client	{ component_id, severity, recommended_action, predicted_rul, anomaly_flag, timestamp }
inject_anomaly	Client → Server	{ type: "vibration" | "thermal" | "rpm" }
advance_time	Client → Server	{ hours: integer }
reset	Client → Server	{}
connection	Client → Server	Automatic on page load; triggers simulation start if not running

MACHINE LEARNING PIPELINE — DETAILED SPECIFICATION

Training Data Generation

10,000 synthetic engine degradation trajectories are generated using the sensor simulation engine with randomised parameters: initial health (95–100%), fault introduction point (100–400 flight hours), fault growth rate (slow/medium/fast), and environmental noise level. Each trajectory runs until health reaches 0% (simulated failure), producing a labelled dataset of (features, true_RUL) pairs across the full degradation lifecycle.

Feature Engineering — 12 Features per Component

Feature ID	Feature Name	Engineering Rationale
F1	rolling_mean_vibration_10	Mean vibration over last 10 ticks — smooths noise, captures trend level
F2	rolling_std_vibration_10	Std deviation of vibration over last 10 ticks — captures instability
F3	vibration_slope	Linear regression slope of vibration over last 20 ticks — rate of change
F4	rolling_mean_temp_10	Mean temperature over last 10 ticks
F5	temp_rise_rate	Rate of temperature increase per flight hour (°C/hr)
F6	rolling_std_temp_10	Std deviation of temperature — captures thermal instability
F7	rpm_drift	Deviation of current RPM from baseline RPM — compressor fouling indicator
F8	vib_temp_correlation	Pearson correlation between vibration and temperature over last 20 ticks
F9	cumulative_fatigue	Running fatigue score from the rule-based engine — physics-informed feature
F10	health_score	Current health percentage — direct state indicator
F11	flight_hour_normalised	Current flight hour normalised to 0–1 range (max 1000 hrs)
F12	anomaly_z_score_max	Maximum z-score across all three sensor streams in last 10 ticks

Model Training & Validation

- Algorithm: Random Forest Regressor — chosen for interpretability, robustness to noisy features, and built-in feature importance output.
- Hyperparameters: n_estimators=100, max_depth=12, min_samples_split=5, random_state=42. Depth-capped intentionally to reflect edge hardware compute constraints.

- Train/test split: 80% training (8,000 trajectories), 20% validation (2,000 trajectories). Stratified by fault type to ensure all failure modes are represented in both splits.
- Validation metrics: MAE ~12–18 flight hours, RMSE ~22–30 flight hours, R^2 ~0.91–0.94 on validation set. Acceptable for a planning tool where maintenance slots are typically scheduled in 48–72 hour windows.
- Serialisation: Trained model saved as aerotwin_rul_model.pkl via joblib. Loaded once at Flask startup and held in memory for inference during the simulation.